

Questions We Expect

Thirty questions judges are likely to ask, the honest answer to each, and the evidence to point at.

Three rules for answering

1. **Answer the question asked**, then stop. Do not pre-empt the follow-up.
2. **Name the evidence.** Every claim here has a file, a command or a number behind it.
3. **Say “we don't know” when we don't.** A gap admitted is survivable; a claim that collapses under one follow-up is not.

Print this. Do not read from it during the presentation.

The question they will definitely ask

Where is the AI in this?

The AI is the thing being *controlled*, not the thing doing the controlling. We built the model hook — three providers, one config line — then measured whether it earned its place. It added up to 4.7 seconds to a decision and changed zero outcomes, so it ships off by default.

Putting a model in the path that gates spending would itself be the vulnerability, in a challenge whose whole point is text trying to talk a decider into approving.

Evidence: 200-run profile, model off 0.172 ms median / model on 2,141 ms. Slide 10.

So you didn't use AI at all?

We did, in two places, both optional and both outside the hot path. An advisor gives a second opinion on whether a cart line is the item the customer described. A clarifier rewords a question our compiler already generated, and rewrites the instruction from the customer's answer.

Both are constrained the same way: the advisor never sees the policy and can only raise doubt, never grant permission. The clarifier changes wording only — code decides when to ask and which answers are acceptable.

Is there any machine learning?

No, and we would rather say so than imply otherwise. Behavioural signals — familiarity, velocity, device novelty — are computed arithmetically from the 4,701 history rows, not learned. At that data scale we think that is the right call: 4,701 rows is not a training set, and a learned model would be less explainable for no measurable gain.

The brief suggests combining five disciplines. We use four of them.

Correctness and method

How do you know it is not hard-coded to the scenarios?

There is a test that rewrites `scenario_id`, `request_id` and `replay_order` on every event and asserts the decisions are unchanged. Nothing in the engine reads any of the three.

Evidence: `test_decision.py`, `test_decisions_do_not_depend_on_scenario_id_or_position`

There is no answer key. How do you know the decisions are right?

We can't prove correctness against ground truth, because the pack deliberately ships none. What we can do is make every verdict defensible: each carries the evidence that produced it, and each decline traces to a rule the customer actually stated.

The strongest independent check we have is that the live API and the offline replay produce identical counts across all 45 purchases. Same answers, both paths.

23 declines out of 45. Isn't that over-blocking?

Every decline names the rule it breached. Two of the supplied attempts exist specifically to catch over-blocking — AU0023, a shop this card has never used but which meets every stated condition, and AU0042, a legitimate re-quote after an earlier decline. Both are approved.

SCEN0001 declines five in a row only because the customer's own CHF 300 seven-day limit was reached. That is the policy working, not the engine being cautious.

What was the hardest part?

Keeping “uncertain” distinct from “no”. Pass and fail are claims about the purchase; uncertain is a claim about us. Collapse it one way and you block ordinary shopping, which the brief counts as a failure. Collapse it the other way and you wave through what nobody checked.

It took four bugs to get right. One of them had an unstated return policy declining, which is not permission but is not a refusal either.

Did you find bugs in your own system?

Eight, and they are listed in the report with how each was caught. Four came from running the supplied data rather than reading it. One came from a round-trip test. One came from first contact with the live API. Two came from using our own interface.

The most instructive: a rolling limit that compared only prior spend, so CHF 299.50 plus 65.50 passed a CHF 300 cap and the *next* order got blamed.

Security and the injection scenario

Could a merchant get a purchase approved by writing the right text?

No, and not because we filter well. The sanitiser returns data and holds no reference to the policy. Facts are extracted by patterns that can only ever emit a number or a token. There is no code path by which merchant text reaches the verdict.

That is a structural property, not a detection rate.

Why not use an LLM to detect prompt injection? That is what they are good at.

Because it inverts the threat model. Using a language model to screen attacker-controlled text means feeding that text to a model whose output then influences a decision. You have moved the attack surface, not removed it.

A regular expression has no such surface. It emits a boolean and nothing else.

What is your false positive rate on injection detection?

Exactly 2 of the 56 supplied cart lines flag, and there is a test asserting that number. Ordinary copy — “Road-running shoe, size 43; returns accepted within 30 days” — passes untouched. Blocking ordinary shopping is a failure too.

Evidence: `test_sanitize.py`, `test_ordinary_product_copy_is_not_flagged`

What if the injection is encoded, or uses invisible characters?

Text is normalised before scanning — NFKC folding plus removal of zero-width joiners, bidirectional overrides and the soft hyphen. There is a test that hides an instruction using zero-width characters and asserts it still fires.

You detected the two injections in the pack. Would you catch a new one?

Honestly: the six signatures describe forms of address, not topics, so they generalise better than a keyword list. But a novel phrasing could evade them.

What protects us there is that evasion gains the attacker nothing. The text cannot change a rule regardless. An undetected injection means a missed signal, not an approved purchase.

How do you stop a lookalike merchant?

Familiarity is matched on `merchant_id`, never on a name, because a name is the one thing an impostor controls. Names are indexed for one purpose only: telling the customer that an unfamiliar shop is wearing a familiar shop's name.

PixelHarbour scores 96% similar to PixelHarbor, which this card has used six times. The genuinely new Circuit and Pine draws no similarity finding at all — which is what proves the check discriminates rather than just fires.

Customer control

How does the customer actually stay in control?

Four ways, and none of them are in the contract. Nothing runs before they confirm. A live policy can only be tightened, never loosened. A paused purchase is not an approval and does not count against a limit. Revocation is immediate and survives a restart.

What happens if the customer never answers?

Nothing is sent. We measured the live API: an unanswered purchase moves to `timed_out` on its own at exactly 120 seconds.

A decline we submitted because nobody replied would be recorded as *the customer's* decision when they never made one — which the guide explicitly forbids. Lapsing is the absence of an answer, not an answer.

Evidence: `measured live, t+0 awaiting_customer, t+120 timed_out`

Why can a policy only be tightened? What if I want to raise my limit?

The platform requires it — a PATCH must keep every existing rule. But the real guarantee is conjunction: every rule ANDs, so an addition can only remove purchases from the allowed set.

Adding “under CHF 200” on top of “under CHF 120” therefore does not raise the limit. 120 still binds. We tell the customer that explicitly rather than let them believe otherwise. To genuinely relax it, you revoke and authorise a new policy.

Why does it ask the customer questions before confirming?

Because some things the sentence does not settle. “A shop I use regularly” has no field in the event — somebody has to decide whether that means one past purchase or five, and that somebody is the customer.

We also raise a gap the instructions never mention but the data does: two orders minutes apart can each sit under a per-order cap while together clearing it.

Engineering and architecture

Why is the UI a separate service?

Viseca said they intend to fold wallet control into the existing *one* app while the decision runs in the backend under a hard latency ceiling. Those are opposite requirements — one is interactive and redeployed freely, the other is a hot path measured in milliseconds.

We kept them as separate deployables from the first commit, and verified by grepping the import graph that the engine never imports from the control plane.

Would this scale?

The engine is a pure function with no I/O, so it scales horizontally with no coordination. Two things would not scale as written: familiarity loads all 4,701 history rows at import and should be a query, and the control session is a JSON file rather than a database.

Both are prototype decisions we would change, not design flaws.

What is your latency headroom really?

Worst measured decision is 0.479 ms against 8,000 ms. Parsing costs more than deciding — 0.744 ms — because we validate twice, against the platform's own schema and then our types.

The eight seconds start when the purchase is *queued*, not when we poll, so the real headroom is always less than eight. That is precisely why the model is off.

What happens if the sandbox goes down mid-run?

The worker keeps polling; a 204 is not the end of a run. A purchase delivered twice gets its saved answer re-sent rather than recomputed, so a retry cannot charge a limit twice. And the decision journal is append-only, so a restart rebuilds spend totals rather than handing the agent a fresh budget.

What we have not built is reconciliation through `/v1/events`. That is on the gap list.

How many tests, and what do they actually cover?

301. They are not coverage-driven — each one pins a specific way the system could go wrong. That money is Decimal and not float. That “unknown” never becomes “yes”. That a broken model gives identical decisions to no model. That answering a paused purchase does not put it back in the inbox.

What would you build next, with another day?

Reconciliation through the events feed, so a network drop mid-run recovers cleanly instead of relying on redelivery. Then replacing the two tuned constants — familiarity at one prior purchase, a three-hour duplicate window — with something derived rather than chosen.

Harder and more sceptical

This is mostly regular expressions. Is that really a solution?

For the part that gates spending, yes, and deliberately. The compiler's output is a security boundary: a model that can be nudged into emitting “under 900” instead of “under 200” is a hole.

It also has to be deterministic. A customer confirming a policy is agreeing to *those* rules, so the same sentence must always compile the same way — and there is a test for that.

The honest trade-off is that regex handles the phrasings we anticipated and misses others. The mitigation is that every extracted rule is shown back before confirmation, so a bad reading is visible rather than silent.

What is the weakest part of this system?

The instruction compiler, for the reason above. We found two real bugs in it during the build: “CHF 300 a week” compiled to a per-order cap, which is a much weaker policy than the customer wrote; and “never spend more than CHF 500” produced no limit at all, because “more than” reads as a minimum.

Both are fixed with regression tests. Both suggest there are others we have not hit.

You are using a fixed threshold for merchant familiarity. Isn't that arbitrary?

Yes. One prior approved purchase is the reading that does not invent a relationship the history does not show, but it is a judgement, not a measurement. The three-hour duplicate window is the same.

What we did instead of hiding it: the compiler explicitly asks the customer to set the familiarity threshold themselves, and flags it as an open question before they confirm.

Your demo only uses the five supplied scenarios. Does it work on anything else?

The policy compiler takes any sentence — you can type one now and watch it compile. The decision engine reads only policy and purchase facts, and a test proves it ignores scenario identity entirely.

What we cannot claim is coverage beyond 45 purchases. We have no other data.

How much of this did you write, versus generate?

Answer this one straight and briefly, in whatever way is true for your team. Do not improvise it here.

If the rules ask about AI assistance, disclosed assistance is a much smaller problem than assistance that looks concealed.

Show me it working on something you have not rehearsed.

Yes — type any instruction into the box and it compiles live. Or run `leash policy "<their sentence>"` in the terminal.

Be ready for it to mis-read an unusual phrasing. If it does, the right response is to point at the rules panel: “that is exactly why we show every extracted rule back before anything is authorised.”

Rehearse this. It is the most likely live-failure moment.

Numbers to have ready

FIGURE	VALUE
Purchases answered on the live API	45
Approved / declined / asked	17 / 23 / 5
Slowest decision, live	2.88 ms
Slowest decision, 200-run profile	0.479 ms
Platform budget	8,000 ms
Parse and validate	0.744 ms
Index 4,701 history rows, once	23.7 ms
Model latency, median	1,191 ms
Decision with model, worst	4,716 ms (59% of budget)
Model cost per call	\$0.000043
Model cost, 1,000 full runs	\$1.93
Decisions changed by the model	none
Tests	301
Injection signatures	6
Cart lines flagged, of 56	2
Policy fields in the closed namespace	19
Lookalike similarity, PixelHarbour	96%
Human answering window	120 s
Platform redelivery	3 s

If you do not know the answer

“I don't know — that's not something we measured.” Then say what you would measure. Judges have seen every variety of bluff.

If a demo breaks live

“Let me show you the offline replay — same engine, same answers.” Then run `make demo`. It needs no network.

If they challenge a decline

Expand the findings panel. Every decline names the rule it breached and the customer's own words that produced it.